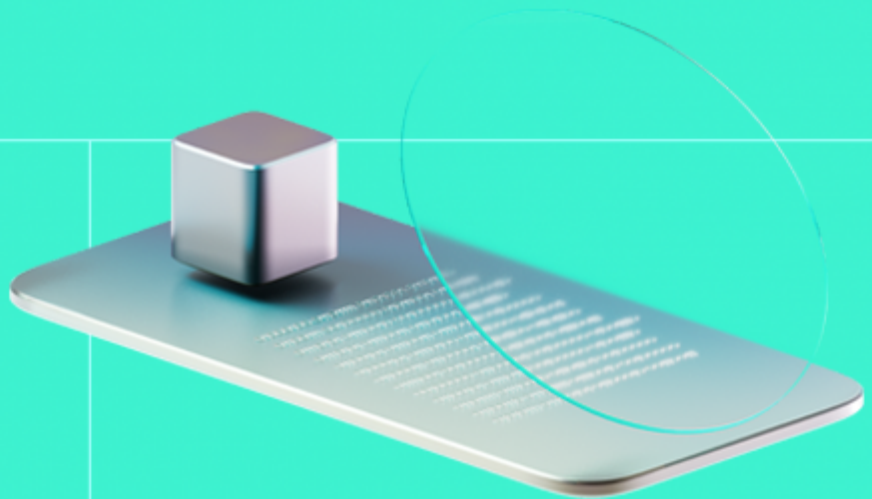




Smart Contract Code Review And Security Analysis Report

Customer: ADI Chain

Date: 25/05/2026



We express our gratitude to the ADI Chain team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

ADI Token is an ERC-20 token deployed on ADI Chain, an EVM-compatible blockchain operated by ADI Foundation. The current contract version (V2) provides a minimal token interface with role-based access control, UUPS proxy upgradeability, and rescue functionality for recovering accidentally sent native and ERC-20 tokens.

Document

Name	Smart Contract Code Review and Security Analysis Report for ADI Chain
Audited By	Kerem Solmaz
Approved By	Seher Saylik
Website	https://www.adi.foundation/
Changelog	21/05/2026 - Preliminary Report
Platform	EVM
Language	Solidity
Tags	Fungible Token, Upgradable
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review

Scope

Repository	https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract
Commit	ae745f4

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

3	0	3	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity Status	
F-2026-17184 - BURNER_ROLE Constant is Unused, With Misleading Storage Slot Comment	Info	Accepted
F-2026-17190 - Rescue Functions Accept Zero Amount and Emit No-Op Events	Info	Accepted
F-2026-17192 - Discrepancy Between initialize NatSpec and On-Chain Token Name, and Absence of Initial Supply	Info	Accepted

Documentation quality

- The project provides a README with an overview of roles, features, and inheritance hierarchy, along with deployment and testing instructions.
- NatSpec documentation is present on all public and external functions. However, no formal technical specification, whitepaper, or architectural document accompanies the codebase.
- Documentation-to-implementation discrepancies were identified and reported.

Code quality

- The codebase is compact and well-organized, following Solidity and OpenZeppelin best practices.
- The contract inherits from OpenZeppelin Contracts Upgradeable v5.3.0 for ERC-20 functionality, access control, and UUPS upgradeability.
- Code style is consistent, and the use of a custom library for zero-address validation demonstrates attention to input safety.
- Issues related to dead code and documentation accuracy were identified and reported.

Test coverage

Code coverage of the project is **91.67%** (branch coverage). Mutation score was not calculated.

- Deployment, initialization, role management, token rescue flows, zero-address checks, event emission, and upgrade flows are covered with tests.
- Negative cases are partially covered, including unauthorized access, zero-address inputs, and invalid burn scenarios.
- Multi-user interaction scenarios and mutation testing are not covered.

Table of Contents

System Overview	6
Privileged Roles	
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	14
Appendix 1. Definitions	15
Severities	15
Potential Risks	15
Appendix 2. Scope	16
Appendix 3. Additional Valuables	17

System Overview

ADI Token is a lightweight ERC-20 contract deployed behind a UUPS proxy on ADI Chain. The current implementation (V2) removes the minting and burning capabilities present in V1 and retains only token transfer functionality, role-based access control, and asset rescue operations. The token was originally minted under V1; V2 is deployed as an implementation upgrade to the existing proxy, preserving the on-chain token state including name, symbol, and total supply.

The **ADI** contract serves as the main entry point and inherits from four base contracts.

ERC20Upgradeable provides standard ERC-20 token operations including transfers and balance queries. **UUPSUpgradeable** enables the proxy upgrade pattern, restricting implementation changes to authorized accounts. **AccessControlUpgradeable** manages role-based permissions through OpenZeppelin's role system. **TokensRescuer** provides internal methods for recovering native (ETH) and ERC-20 tokens that are accidentally sent to the contract address.

The **TokensRescuer** abstract contract implements two rescue operations. The native token rescue function validates the receiver address through the **ZeroAddrChecker** library and transfers the specified amount using OpenZeppelin's `Address.sendValue`. The ERC-20 rescue function validates both the token and receiver addresses and executes a safe transfer using OpenZeppelin's **SafeERC20**. Both functions emit corresponding events upon successful execution. The contract reserves 50 storage slots via a gap array to ensure upgrade-safe storage layout.

The **ZeroAddrChecker** library provides a single validation function that reverts with a custom error when a zero address is supplied, used throughout the rescue operations to prevent transfers to the null address.

The `initialize` function sets the token name and symbol and grants `DEFAULT_ADMIN_ROLE` to the deployer. Under the documented operational model, this function is not expected to execute against the production proxy, as the contract is deployed strictly as an implementation upgrade preserving existing proxy state.

Privileged roles

- **DEFAULT_ADMIN_ROLE:** Granted to the deployer during initialization. This role can recover native tokens and ERC-20 tokens from the contract via the rescue functions, directing arbitrary amounts to any non-zero address. As the default admin role in OpenZeppelin's AccessControl system, this role can also grant and revoke all other roles, including `UPGRADER_ROLE`. Compromise of this role would enable draining any tokens held by the contract and granting upgrade authority to an attacker-controlled address. No timelock or multisig requirement is enforced on-chain for this role.
- **UPGRADER_ROLE:** Authorized to upgrade the contract implementation via the UUPS proxy pattern. An account with this role can replace the entire contract logic, gaining unrestricted control over token state, balances, and all contract behavior. This role is not granted during initialization and must be explicitly assigned by the `DEFAULT_ADMIN_ROLE` holder. Compromise or misuse of this role represents the highest-impact risk in the system. No timelock, governance vote, or mandatory review period constrains the upgrade process.
- **BURNER_ROLE:** Defined as a constant in the contract but not used in V2. No functions reference this role, and no accounts are granted this role during initialization. It is retained from V1 as a

storage-layout artifact (though as a constant it does not occupy a storage slot). This role has no functional impact in the current implementation.

Potential Risks

- **Single Points of Failure and Control:** The **ADI** contract relies on role-based access control where the `DEFAULT_ADMIN_ROLE` holder has the authority to rescue all native and ERC-20 tokens held by the contract, grant and revoke all roles including `UPGRADER_ROLE`, and indirectly control the upgrade path of the entire token implementation. Concentration of these capabilities in a single account — without on-chain multisig or timelock enforcement — creates a single point of failure. Compromise of this account would grant an attacker full operational and upgrade control over the ADI token.
- **Administrative Key Control Risks:** The `UPGRADER_ROLE` in the **ADI** contract authorizes replacement of the entire contract implementation through the UUPS proxy mechanism. The `DEFAULT_ADMIN_ROLE` can assign this upgrade authority to any address at any time. Together, these two roles form a chain of administrative key control where a single compromised key (the default admin) can escalate to full implementation replacement. No separation of duties, multisig requirement, or community governance mechanism is enforced at the contract level to constrain this authority.
- **Flexibility and Risk in Contract Upgrades:** The **ADI** contract is upgradable via the UUPS proxy pattern, allowing the `UPGRADER_ROLE` holder to replace the contract logic at any time. While this provides flexibility to address issues and evolve the token functionality — as demonstrated by the V1-to-V2 upgrade that removed minting and burning — it also means that the token's entire behavior, including balance accounting and transfer logic, can be changed without token holder consent. A malicious or compromised upgrade could alter balances, disable transfers, or introduce backdoors.
- **Absence of Upgrade Window Constraints:** The UUPS upgrade function in the **ADI** contract executes immediately upon authorization by the `UPGRADER_ROLE` holder, with no mandatory delay, announcement period, or governance review. Token holders and integrating protocols have no opportunity to review proposed implementation changes or take protective action before new logic takes effect. Introducing a timelock or governance mechanism before upgrades would provide a safety window for stakeholders to evaluate changes.
- **Dependency on Unaudited External Libraries:** The **ADI** contract depends on OpenZeppelin Contracts Upgradeable v5.3.0 for ERC-20 functionality, access control, UUPS upgradeability, SafeERC20 transfers, and address utilities. While OpenZeppelin is a widely adopted and previously audited framework, the specific version integrated has not been independently verified as part of this engagement. Any undiscovered vulnerabilities in these base contracts would directly affect the **ADI** token's transfer mechanics, access control enforcement, and upgrade authorization.

Findings

Vulnerability Details

F-2026-17184 - BURNER_ROLE Constant is Unused, With Misleading Storage Slot Comment - Info

Description: The `BURNER_ROLE` constant is retained from V1 with a `@dev` comment stating

```
"Booked storage slot (v1 version)."
```

This comment is technically inaccurate: Solidity `constant` variables are inlined at compile time and do not occupy storage slots. **ADI** contains no `burn` function, so the role is inert.

Assets:

- `contracts/ADl.sol` [<https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Remove `BURNER_ROLE` entirely, since it is not used and only inflates contract bytecode.

Resolution: Accepted, the client stated:

Acknowledged. The team accepts this finding as informational. No production upgrade will be performed, as the unused constant has no functional impact in the currently deployed V2

implementation: no V2 function references BURNER_ROLE and V2 exposes no burn functionality. This may be cleaned up in a future implementation if another upgrade is scheduled.

F-2026-17190 - Rescue Functions Accept Zero Amount and Emit No-Op Events - Info

Description: Both `_rescueNativeToken` and `_rescueERC20Token` succeed when called with `amount = 0`. The calls transfer nothing but still emit `NativeTokenRescue` or `ERC20TokenRescue` events with a zero value, wasting gas on the external call and event emission. While not exploitable, the unnecessary logs add noise to the contract's event history.

Assets:

- contracts/extensions/TokensRescuer.sol
[<https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 3/5
Exploitability: Dependent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Add zero amount checkers before transfers:

```
if (amount == 0) revert ZeroAmount();
```

Resolution: Accepted, the client stated:

Acknowledged. The team accepts this finding as informational. No production upgrade will be performed, as zero-amount rescue calls do not create asset-loss risk, privilege escalation, or unexpected state modification risk. The behavior is limited to potential gas waste and event-log noise on admin-only rescue functions. Operational procedures will avoid zero-amount rescue calls, and this may be cleaned up in a future implementation if another upgrade is scheduled.

F-2026-17192 - Discrepancy Between initialize NatSpec and On-Chain Token Name, and Absence of Initial Supply - Info

Description: In **ADI**, the NatSpec attached to the `initialize` function documents the token as being created with the name `ADI Token`:

```
/**
 * @notice Initializes the ADI token with the name `ADI Token` and symbol
 *         `ADI`. Also grants the `DEFAULT_ADMIN_ROLE` to the `msg.sender`.
 */
function initialize() external initializer {
    __ERC20_init_unchained("ADI", "ADI");
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}
```

The underlying call passes `"ADI"` as the name argument rather than `"ADI Token"`, so the documented and on-chain values diverge. The intended deployment path installs this implementation as an upgrade behind an existing proxy whose ERC20 storage already carries the historical name, which makes the divergence invisible at runtime under normal operation; however, the documentation no longer describes what `initialize` actually writes if it is ever exercised against a freshly created proxy.

Additionally, the function performs no initial mint and the contract exposes no mint entry point, so an `initialize`-driven setup would leave `totalSupply()` at zero with no on-chain remediation short of an implementation upgrade. The combination is unused under the documented upgrade-only operational model, but the prose contract and the executable contract are not aligned.

Assets:

- `contracts/ADI.sol` [<https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract>]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity:

Info

Recommendations**Remediation:**

Align the NatSpec with the value actually written, or update the call to match the documented name. If the deployment model is upgrade-only, document this explicitly.

Resolution:

Accepted, the client stated:

Acknowledged. The team accepts this finding as informational. No production upgrade will be performed, as the deployed V2 implementation was installed as an upgrade behind an already initialized V1 proxy. The production proxy preserves the existing token state, including name, symbol, balances, and total supply, and the V2 initializer is not part of the production execution path. This documentation mismatch may be corrected in a future implementation if another upgrade is scheduled.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract
Commit	ae745f4f3e2457cfe7fc8e1c378f2eaeff466914
Whitepaper	https://docs.adi.foundation/whitepaper
Requirements	README.md
Technical Requirements	README.md

Asset	Type
contracts/ADI.sol [https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract]	Smart Contract
contracts/extensions/TokensRescuer.sol [https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract]	Smart Contract
contracts/extensions/ZeroAddrChecker.sol [https://gitlab.sre.ideasoft.io/adi-foundation/adi-chain/adi-token-smart-contract]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.